LEMBAR KERJA PRAKTIKUM

Pemrograman Mobile

Pertemuan 7 HTTP Networking dan Konsumsi REST API

Mata Kuliah	Pemrograman Mobile
Pertemuan	Pertemuan 7
Topik	HTTP Networking dan Konsumsi REST API
Studi Kasus Project	TokoKita, mengambil data produk sungguhan dari REST API
Framework	Flutter (Dart)
Alokasi Waktu	170 menit (3 SKS)
Nama Mahasiswa	Dimas Panca Pamungkas
NIM	3337240063

A. Tujuan Praktikum

- Mahasiswa mampu menggunakan package http untuk melakukan request ke REST API.
- Mahasiswa mampu melakukan request GET dan POST beserta penanganan response-nya.
- Mahasiswa mampu melakukan parsing JSON menggunakan jsonDecode dan jsonEncode serta mengadaptasi model class dengan fromJson dan toJson.
- Mahasiswa mampu menampilkan data asynchronous menggunakan async/await dan FutureBuilder.
- Mahasiswa mampu menerapkan penanganan error dasar (try-catch, pengecekan status code) pada proses networking.

B. Alat dan Bahan

- Laptop atau PC dengan proyek tokokita beserta HomePage dan class Product dari pertemuan sebelumnya
- Visual Studio Code dan Flutter SDK
- Chrome dengan tampilan mobile sebagai pengganti emulator, koneksi internet aktif
- Package http yang sudah ada di pubspec.yaml
- REST API publik Fake Store API (fakestoreapi.com/products)

C. Dasar Teori Singkat

Sampai pertemuan sebelumnya data produk TokoKita masih dummy yang ditulis langsung di kode. Supaya data bisa berubah tanpa rilis ulang aplikasi, data diambil dari REST API memakai package http. Karena butuh waktu, request berjalan asynchronous dengan Future dan async/await, lalu FutureBuilder membangun tampilan sesuai status loading, berhasil, atau gagal. Respons JSON diubah dengan jsonDecode lalu dipetakan ke class Product lewat fromJson, dan status code dicek dulu sebelum data diproses.

D. Langkah Kerja

Langkah 1. Setup Package http

- Memastikan package http (versi 1.6.0) sudah ada di pubspec.yaml lalu menjalankan flutter pub get.
- Membuat lib/services/product_service.dart untuk semua fungsi pengambilan data produk.
- Menambahkan izin INTERNET di AndroidManifest.xml supaya aplikasi Android boleh mengakses jaringan.

Langkah 2. Request GET Data Produk

- Membuat request GET ke fakestoreapi.com/products memakai http.get dengan async/await.
- Mencetak status code ke console memakai print untuk memastikan data diterima.

Langkah 3. Menyesuaikan Model Product dan Parsing JSON

- Menambahkan factory Product.fromJson dan method toJson pada class Product.

- Mengubah response.body dengan jsonDecode lalu memetakannya menjadi List<Product> pada fetchProducts.

Langkah 4. Menampilkan Data Asli dengan FutureBuilder

- Mengubah HomePage menjadi StatefulWidget supaya future bisa disimpan dan dipanggil ulang.
- Memakai FutureBuilder dengan ListView.builder dan ProductCard yang lama. ProductCard hanya ditambah Image.network untuk menampilkan gambar dari URL.
- Menampilkan CircularProgressIndicator saat data masih dimuat.

Langkah 5. Penanganan Error

- Membungkus request dengan try-catch dan mengecek response.statusCode sebelum parsing.
- Mengubah URL menjadi productz secara sengaja untuk mensimulasikan kegagalan, hasilnya server membalas 404.
- Menambahkan pesan Gagal memuat produk, coba lagi beserta tombol Coba Lagi yang memanggil ulang fetchProducts.

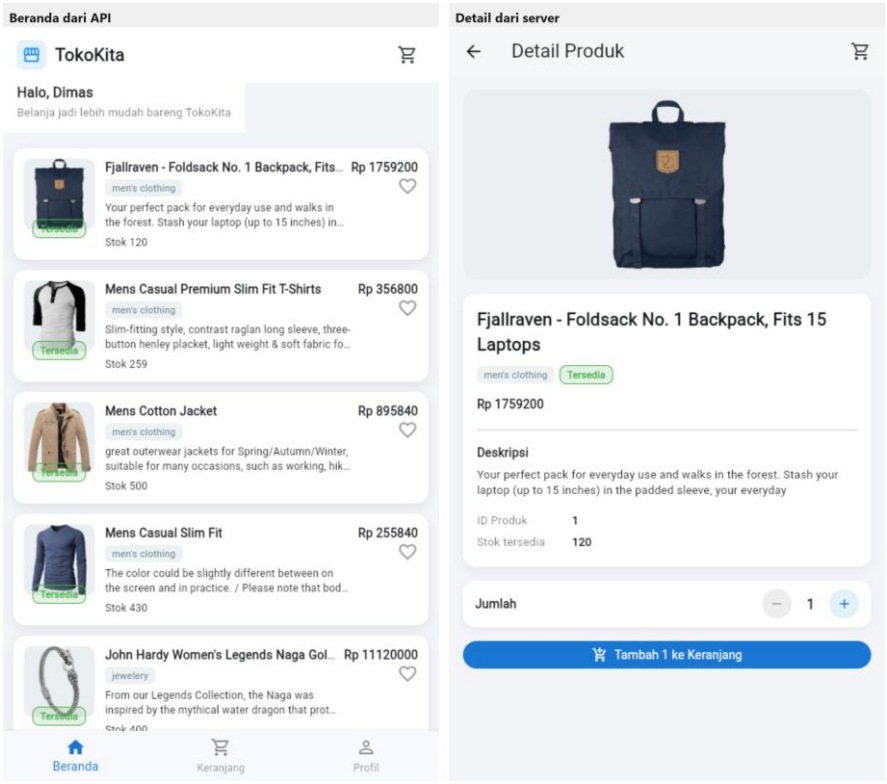
E. Hasil Pengamatan

Hasil pengujian request pada tabel berikut.

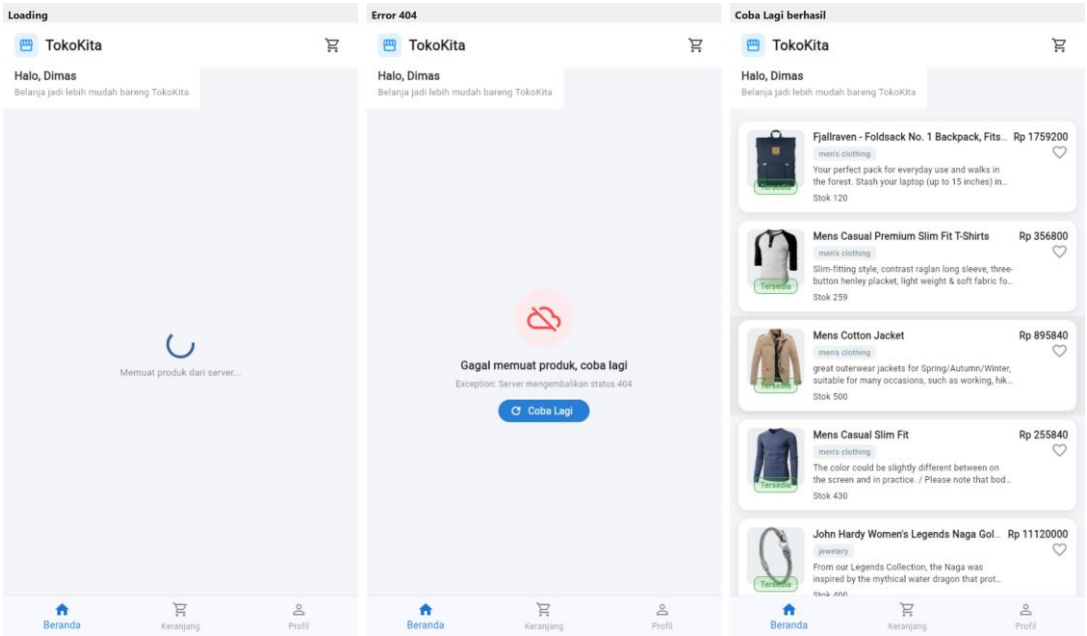
No.	Jenis Request atau Skenario	Status Code atau Hasil	Screenshot
1	GET data produk dari Fake Store API	Status 200. Data produk diterima dalam bentuk JSON array dan tercatat di log terminal.	Ya
2	Parsing JSON ke Product.fromJson	Semua item berhasil menjadi objek Product. Harga dikali kurs 16000 dan rating count dipakai sebagai stok.	Ya
3	Tampilan produk asli via FutureBuilder	Indikator loading muncul lebih dulu, lalu daftar produk tampil lengkap dengan gambar dari server.	Ya
4	Simulasi error (URL salah)	Server membalas 404. Layar menampilkan pesan gagal memuat produk dan tombol Coba Lagi.	Ya
5	Tombol Coba Lagi setelah error	URL dibenarkan lalu tombol ditekan. Loading muncul lagi, kemudian daftar produk tampil dengan status 200.	Ya

Bukti tampilan aplikasi (mobile view)

1. Beranda dari API dan detail produk dari server



2. Tampilan loading, error 404, dan Coba Lagi berhasil



3. fetchProducts dengan try-catch dan cek status code

```dart
// services/product_service.dart
Future<List<Product>> fetchProducts() async {
  final url = Uri.parse('$_baseUrl/products');
  try {
    final response = await http.get(url);
    if (response.statusCode != 200) {
      throw Exception(
        'Server mengembalikan status ${response.statusCode}');
    }
    final data = jsonDecode(response.body) as List<dynamic>;
    return data
        .map((e) => Product.fromJson(e as Map<String, dynamic>))
        .toList();
  } catch (e) {
    rethrow;
  }
}
```

4. Product.fromJson

```dart
// models/product.dart
factory Product.fromJson(Map<String, dynamic> json) {
  final rating = json['rating'];
  final count = rating is Map
      ? (rating['count'] as num?)?.toInt() ?? 0 : 0;
  final hargaUsd = (json['price'] as num?)?.toDouble() ?? 0;
  return Product(
    id: json['id'].toString(),
    name: json['title']?.toString() ?? 'Tanpa nama',
    price: hargaUsd * kursUsdKeRupiah,
    imageUrl: json['image']?.toString() ?? '',
    category: json['category']?.toString() ?? 'lainnya',
    stock: count,
    description: json['description']?.toString(),
  );
}
```

5. FutureBuilder dan RefreshIndicator pada HomePage

```dart
// screens/home_page.dart
FutureBuilder<List<Product>>(
  future: _future,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const _Loading();
    }
    if (snapshot.hasError) {
      return _ErrorView(onCobaLagi: _cobaLagi, ...);
    }
    return RefreshIndicator(
      onRefresh: _reload,
      child: ListView.builder(...),
    );
  },
)
```

6. Log terminal

```
# log terminal flutter run
GET https://fakestoreapi.com/products -> 200
GET https://fakestoreapi.com/products/1 -> 200
GET https://fakestoreapi.com/productz -> 404
fetchProducts error: Exception: Server mengembalikan status 404
GET https://fakestoreapi.com/products -> 200
```

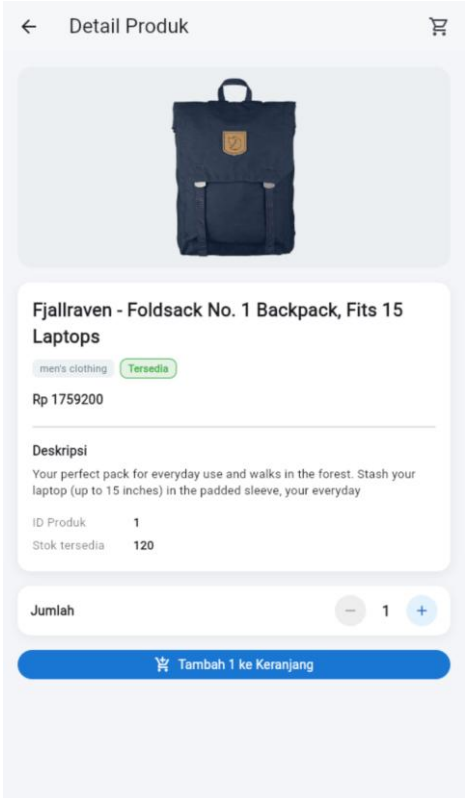
Cara menjalankan aplikasi

Jalankan flutter pub get lalu flutter run -d chrome dan aktifkan tampilan mobile. Pastikan internet menyala karena data diambil dari server.

F. Tugas Mandiri

1. Tambahkan function Future<Product> fetchProductDetail(int id) yang mengambil detail satu produk dari endpoint products/{id}, lalu gunakan pada ProductDetailPage sebagai alternatif data yang dikirim lewat navigasi.

Jawaban. Saya menambahkan fetchProductDetail di ProductService yang memanggil products/{id}. ProductDetailPage memanggilnya di initState, jadi data dari navigasi dipakai dulu lalu diganti hasil server setelah selesai. Kalau request gagal, halaman tetap memakai data dari navigasi. Log mencatat GET products/1 dengan status 200 saat produk pertama dibuka.



2. Terapkan pull-to-refresh (RefreshIndicator) pada HomePage agar pengguna dapat memuat ulang data produk terbaru.

Jawaban. Daftar produk saya bungkus dengan RefreshIndicator yang memanggil _reload. Selama request berjalan, daftar lama tetap tampil dan baru diganti setelah data baru datang. Tiap kali layar ditarik ke bawah, log mencatat satu GET products tambahan dengan status 200.

3. Jelaskan alur lengkap proses request, parsing, dan tampil data pada HomePage TokoKita, dari pemanggilan fetchProducts() hingga data muncul sebagai ProductCard.

Jawaban.

1. initState di HomePage memanggil fetchProducts dan menyimpan Future-nya di _future.
2. FutureBuilder membaca _future. Selama belum selesai, layar menampilkan indikator loading.
3. fetchProducts mengirim http.get ke API lalu menunggu response dengan await.
4. Status code dicek. Kalau bukan 200, exception dilempar dan FutureBuilder masuk ke tampilan error.
5. Kalau 200, body diubah dengan jsonDecode lalu tiap item dipetakan ke Product lewat fromJson.
6. List<Product> dikembalikan ke FutureBuilder, ListView.builder membangun satu ProductCard untuk tiap produk.

4. Bandingkan struktur data JSON dari Fake Store API dengan struktur class Product yang dirancang pada Pertemuan 2, sebutkan penyesuaian apa saja yang diperlukan pada method fromJson.

Jawaban. Nama field dan tipe datanya tidak sama persis, jadi fromJson perlu memetakan satu per satu seperti pada tabel berikut.

Field API	Properti Product	Penyesuaian
id (angka)	id (String)	Diubah dengan toString karena Product memakai String.
title	name	Hanya ganti nama field.
price (dolar)	price	Dikali kurs 16000 supaya tampil dalam rupiah.
image	imageUrl	Ganti nama field, dipakai oleh Image.network.
category	category	Tetap, tetapi isinya berbahasa Inggris seperti men's clothing.
description	description	Langsung dipakai karena sudah nullable.
rating.count	stock	API tidak punya stok, jadi jumlah rating dipakai sebagai pengganti.

5. Menurut pemahaman Anda, mengapa pengecekan response.statusCode tetap diperlukan meskipun request tidak menghasilkan exception? Kaitkan dengan kemungkinan respons error dari server seperti 404 atau 500 pada API produk TokoKita.

Jawaban. Menurut saya pengecekan ini tetap wajib karena http.get hanya melempar exception kalau koneksinya gagal, misalnya internet mati. Kalau server membalas 404 atau 500, request tetap dianggap berhasil dan tidak masuk ke catch. Tanpa cek status, body error ikut di-decode lalu bisa bikin aplikasi crash atau menampilkan data kosong. Saat URL saya ubah menjadi productz, server membalas 404 dan tidak ada exception dari http, jadi hanya pengecekan statusCode yang menangkapnya dan memunculkan pesan gagal memuat produk.

G. Format Pengumpulan

- Laporan dikumpulkan dalam format PDF.
- Format nama file Praktikum7_NIM_NamaMahasiswa.pdf
- Dikumpulkan paling lambat sebelum pertemuan berikutnya dimulai.